

RK3562 多核异构应用



主题	RK3562 多核异构应用
文档号	1.0
创建时间	2025-04-17
最后修改	2025-04-17
版本号	1.0
文件名	RK3562 多核异构应用 V1.0.pdf
文件格式	Portable Document Format

阅前须知

声明

北京合众恒跃科技有限公司保留随时对其产品进行修正、改进和完善的权利，客户在下单前应获取相关信息的最新版本，并验证这些信息是正确的。本文档一切解释权归北京合众恒跃科技有限公司所有。

简介

北京合众恒跃科技有限公司位于中关村科技园区，公司主要从事嵌入式产品的设计、研发、生产、销售等业务；公司的核心竞争力是对嵌入式产品精准的设计及实现能力，成本控制能力和产品品质保障能力；公司主干研发人员均有多年嵌入式产品开发经验和软硬件技术积累，服务于音视频、图像识别、电力等多个应用领域，公司研发设计的视频转码卡、图像识别卡、电力保护装置等产品在业内享有一定的知名度。

联系方式

北京合众恒跃科技有限公司

电话：010-62129511

邮编：102208

技术支持邮箱：support@hzhytech.com

地址：北京市海淀区安宁庄后街南 1 号 A 区一层 1020 号



官方微信公众号



官方企业微信



合众嵌入式官方店铺



合众 AI 官方店铺

修改记录

版本号	日期	修改人	备注
1.0	2025-04-17	马文轩	初始版本

目录

阅前须知	2
声明	2
简介	2
联系方式	2
修改记录	3
目录	4
第 1 章 多核异构案例介绍	5
1.1 AP+AP 案例：电力继电保护装置	5
第 2 章 AP+AP 系统配置指南	6
2.1 系统运行流程介绍	6
2.2 编译项配置方法	6
2.2.1 增加 Uboot 的 AMP 支持	6
2.2.2 增加内核的 AMP 支持	8
2.3 设备树修改说明	9
2.4 AMP 分区划分	9
2.5 调试串口配置方法	9
2.6 系统编译与部署流程	12
2.7 异构系统启动与测试	14
2.8 核间通信配置	16
2.8.1 增加 Kernel 的 rpmsg 支持	18
2.8.2 增加 RT-Thread 的 rpmsg 支持	18

第 1 章 多核异构案例介绍

1.1 AP+AP 案例：电力继电保护装置

在电力继电保护装置中，既对系统的实时性有要求，例如对各种电气量进行实时采集和数据分析、对保护控制信号进行实时响应等。又对系统的丰富性有要求，需要使用复杂的软件功能和硬件外设，例如显示设备、USB 设备、以太网设备等。使用瑞芯微多核异构系统，将传统平台两套系统合二为一，一套板卡就能同时独立运行 Linux 系统和实时系统，实现上述所有功能。

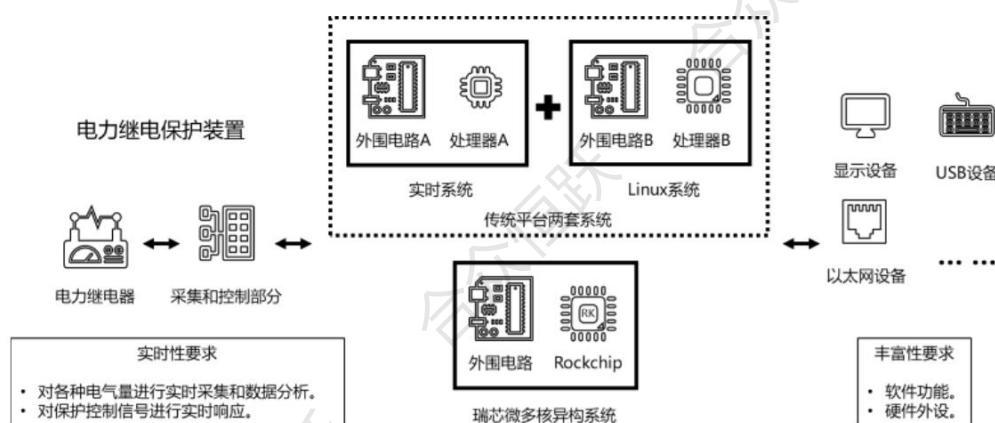


图 1.1-1 系统架构

第 2 章 AP+AP 系统配置指南

2.1 系统运行流程介绍

在 Kernel + RT-Thread / HAL 的启动方案中，默认将 RT-Thread 固件运行于 CPU3。系统启动时，Bootloader 首先在 CPU0 上运行，并加载 U-Boot。在 U-Boot 阶段，系统会读取 AMP 固件并启动 CPU3 运行 RT-Thread。随后，U-Boot 继续在 CPU0 上运行，加载并启动 Linux 内核。在内核启动过程中，CPU1 和 CPU2 被依次唤醒并加入 Linux SMP 系统。

下图展示了整体启动流程框图：

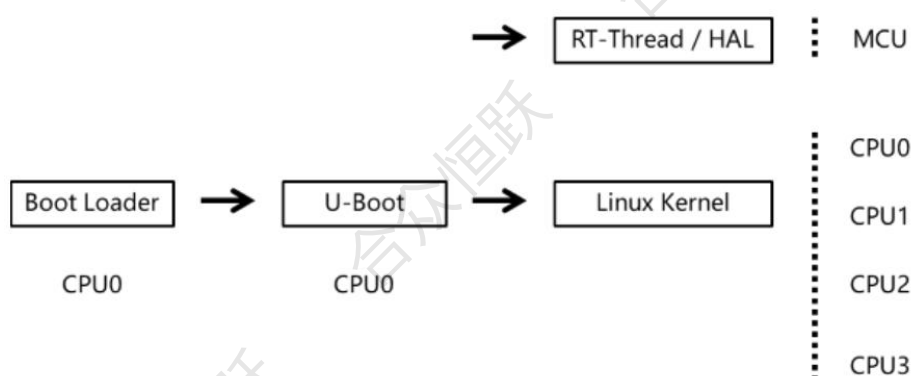


图 2.1-1 流程框图

2.2 编译项配置方法

2.2.1 增加 Uboot 的 AMP 支持

在 uboot 目录中执行 `make ARCH=arm64 menuconfig` 开启配置界面。

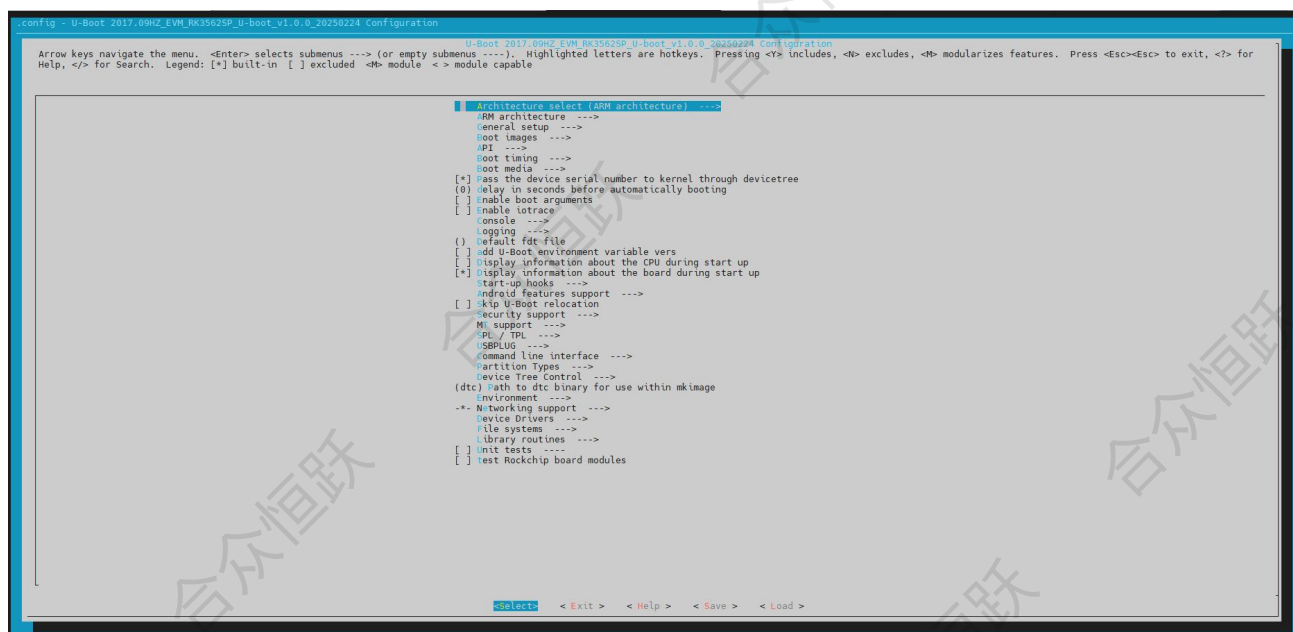


图 2.2-1 配置界面

在配置中搜索 CONFIG_AMP 和 CONFIG_ROCKCHIP_AMP 选项，并将两个选项选中。

CONFIG_AMP=y

```
CONFIG_AMP:
This support Asymmetric Multi-Processing, cpus can run on different
firmware.

Symbol: AMP [=y]
Type : boolean
Prompt: Enable AMP drivers using Driver Model
Location:
-> Device Drivers
Defined at drivers/cpu/Kconfig:10
```

图 2.2-2 修改配置

CONFIG_ROCKCHIP_AMP=y

```
Symbol: ROCKCHIP_AMP [=y]
Type : boolean
Prompt: Enable Rockchip AMP driver
Location:
-> Device Drivers
(1) -> Enable AMP drivers using Driver Model (AMP [=y])
Defined at drivers/cpu/Kconfig:16
Depends on: AMP [=y] && ROCKCHIP_SMCCC [=y] && RKIMG_BOOTLOADER [=y]
```

图 2.2-3 修改配置

完成上述配置修改后，系统会生成临时配置文件 .config。

接着，执行以下命令将该临时配置保存为 defconfig 格式：

make ARCH=arm64 savedefconfig

保存成功后，执行以下命令将生成的 defconfig 拷贝至指定目录，以便作为持久化配置使用：


```
cp defconfig configs/rk3562_defconfig
```

2.2.2 增加内核的 AMP 支持

在 kernel 目录中执行以命令开启内核的配置界面。

```
make ARCH=arm64 savedefconfig
```

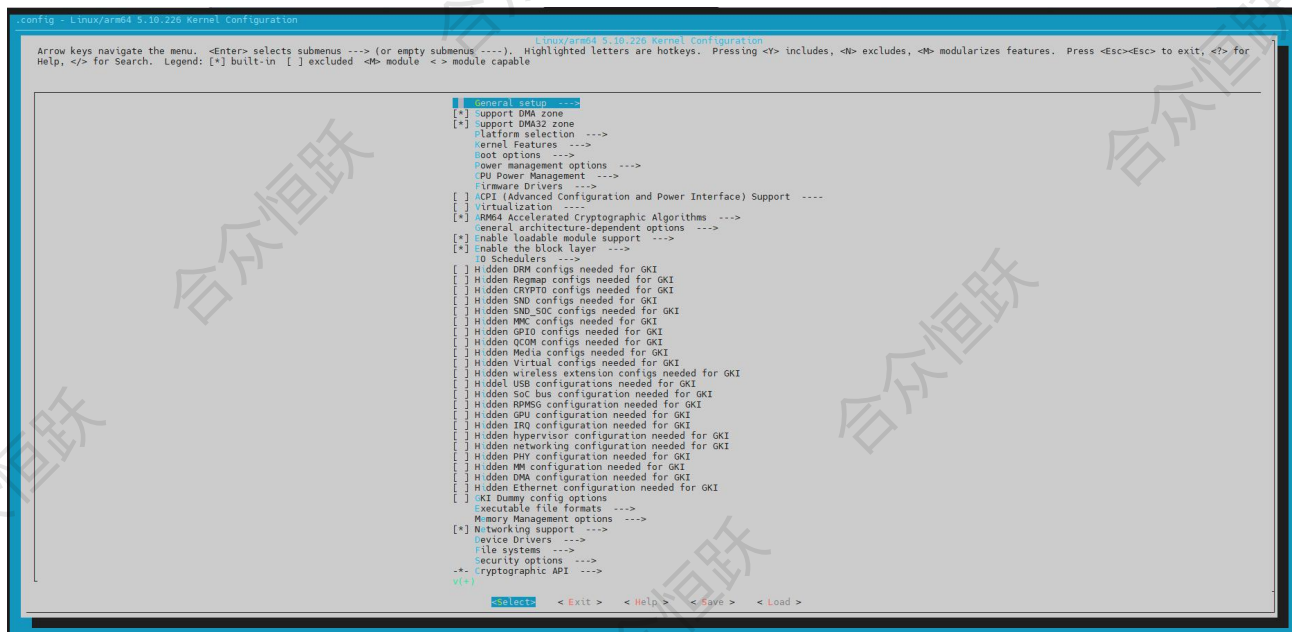


图 2.2-4 配置界面

在配置界面中搜索 CONFIG_ROCKCHIP_AMP 选项，并将该选项选中。

```
CONFIG_ROCKCHIP_AMP = y
```

```
CONFIG_ROCKCHIP_AMP:
Say y here to enable Rockchip AMP support.
This option protects resources used by AMP.

Symbol: ROCKCHIP_AMP [=y]
Type : tristate
Defined at drivers/soc/rockchip/Kconfig:33
Prompt: Rockchip AMP support
Depends on: ARCH_ROCKCHIP [=y] || COMPILE_TEST [=n]
Location:
-> Device Drivers
-> SOC (System On Chip) specific Drivers
-> Rockchip SoC drivers
```

图 2.2-5 修改配置

完成上述配置修改后，系统会生成临时配置文件 .config。

接着，执行以下命令将该临时配置保存为 defconfig 格式：

```
make ARCH=arm64 savedefconfig
```

保存成功后，执行以下命令将生成的 defconfig 拷贝至指定目录，以便作为持久化配置使用：


```
cp defconfig configs/rk3562_defconfig
```

2.3 设备树修改说明

打开设备树文件，并增加如下图 2.3-1 所示节点：

```
vim kernel/arch/arm64/boot/dts/rockchip/HZ-EVM-RK3562-mini.dts
```

```
17 #define CPU_GET_AFFINITY(cpu, cluster) ((cpu) << 0 | ((cluster) << 8))
18
19 &arm_pmu {
20     interrupt-affinity = <&cpu0>, <&cpu1>, <&cpu2>;
21 };
22
23 /delete-node/ &cpu3;
24
```

图 2.3-1 增加设备树节点

2.4 AMP 分区划分

分区表中默认没有 AMP 分区，需要手动添加一个 AMP 分区，新增的 AMP 分区作用时保存 RT-Thread 系统的固件。

执行以下命名添加一个 2MB 大小的 AMP 分区。

```
./build.sh insert-part:4:amp:2M
```

添加分区成功后，执行以下命令查看分区表。

```
./build.sh list-parts
```

```
=====
Partition table
=====
1:      uboot at 0x00004000 size=0x00002000(4M)
2:      misc at 0x00006000 size=0x00002000(4M)
3:      boot at 0x00008000 size=0x00020000(64M)
4:      amp at 0x00028000 size=0x00001000(2M)
5:      recovery at 0x00029000 size=0x00040000(128M)
6:      backup at 0x00069000 size=0x00010000(32M)
7:      oem at 0x00079000 size=0x00040000(128M)
8:      userdata at 0x000b9000 size=0x00200000(1G)
9:      rootfs at 0x002b9000 size=-(grow)
```

图 2.4-1 查看分区表

2.5 调试串口配置方法

注：本案例使用 HZ-RK3562_MiniEVM 开发板，其它板卡根据底板资源分配修改，修改方式相同。

在设备树中修改调试串口配置为 UART7，在这里也需要同步修改调试串口的配置。切换到 rtos/bsp/rockchip/rk3562-32 目录，执行 `scons - menuconfig` 开启配置界面。

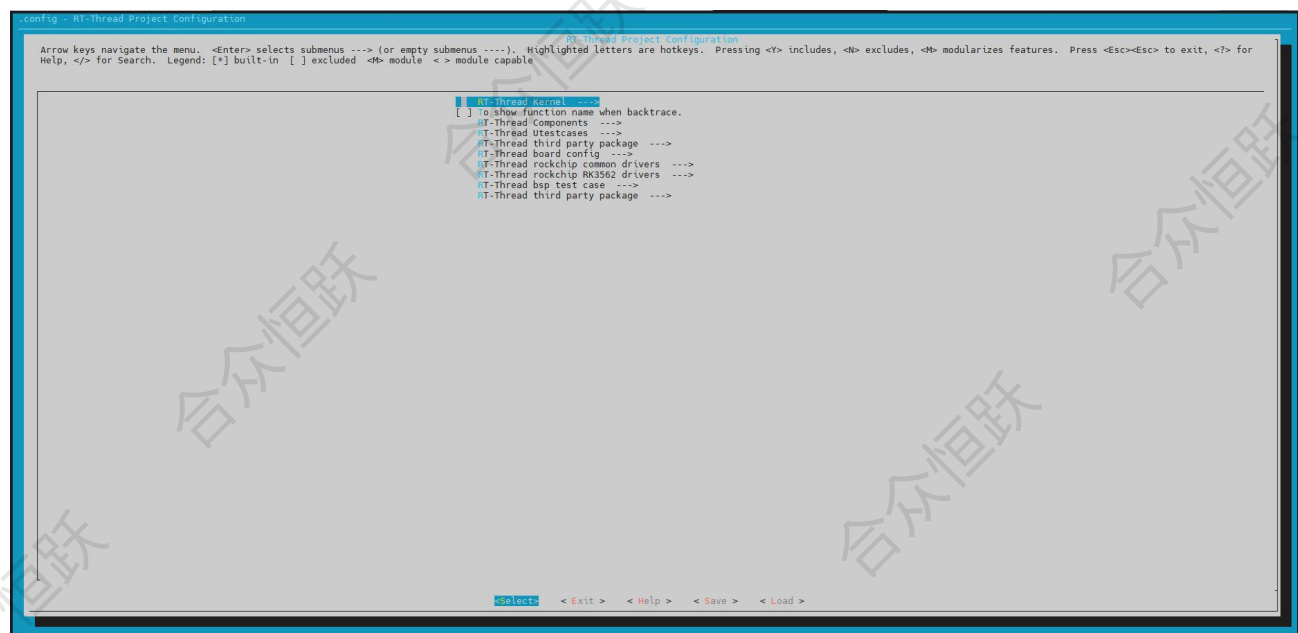


图 2.5-1 配置界面

在配置界面中搜索 RT_CONSOLE_DEVICE_NAME，并将配置修改为 uart7。

```
There is no help available for this option.
Symbol: RT_CONSOLE_DEVICE_NAME [=uart7]
Type : string
Prompt: the device name for console
Location:
-> RT-Thread Kernel
-> Kernel Device Object
-> Using console for rt_kprintf (RT_USING_CONSOLE [=y])
Defined at ../../../../src/Kconfig:441
Depends on: RT_USING_CONSOLE [=y]
```

图 2.5-2 修改配置

搜索 RT_USING_UART7 选项，并使能 UART7。

```
There is no help available for this option.
Symbol: RT_USING_UART7 [=y]
Type : boolean
Prompt: Enable UART7
Location:
-> RT-Thread rockchip RK3562 drivers
-> Enable UART
-> Enable UART (RT_USING_UART [=y])
Defined at ./driver/Kconfig:151
Depends on: RT_USING_UART [=y]
```

图 2.5-3 修改配置

搜索 RT_USING_UART0，RT-Thread 默认开启 UART0，这里需要关闭。

```
There is no help available for this option.
Symbol: RT_USING_UART0 [=n]
Type : boolean
Prompt: Enable UART0
Location:
-> RT-Thread rockchip RK3562 drivers
-> Enable UART
-> Enable UART (RT_USING_UART [=y])
Defined at ./driver/Kconfig:123
Depends on: RT_USING_UART [=y]
```

图 2.5-4 修改配置

RT-Thread 使用 SCons 进行配置时，同样会生成 .config 文件。

但与 Linux 使用 menuconfig 后需执行 `make ARCH=arm64 savedefconfig` 保存配置的方式不同，RT-Thread 无需该步骤。

若需保存当前配置为持久化配置，直接执行以下命令即可：

```
cp .config board/rk3562_evb1_lp4x/amp_defconfig
```

将调试串口由 UART0 修改为 UART7 后，还需新增对应的配置函数。RK 原厂默认支持 UART7_M1 模式，而本项目需使用 UART7_M0，因此需要手动添加 UART7_M0 的 IOMUX 配置函数。

请在以下位置添加和声明函数：

- 在 `rtos/bsp/rockchip/rk3562-32/board/common/iomux_base.c` 文件中新增如下函数定义：

```
cp .config board/rk3562_evb1_lp4x/amp_defconfig
```

将调试串口由 UART0 修改为 UART7 后，还需新增对应的配置函数。RK 原厂默认支持 UART7_M1 模式，而本项目需使用 UART7_M0，因此需要手动添加 UART7_M0 的 IOMUX 配置函数。

请在以下位置添加和声明函数：

- 在 `rtos/bsp/rockchip/rk3562-32/board/common/iomux_base.c` 文件中新增如下函数定义：

```
24
25 #ifdef RT_USING_UART7
26 RT_WEAK void uart7_m1_iomux_config(void)
27 {
28     HAL_PINCTRL_SetIOMUX(GPIO_BANK1,
29                           GPIO_PIN_B3,
30                           PIN_CONFIG_MUX_FUNC3);
31     HAL_PINCTRL_SetIOMUX(GPIO_BANK1,
32                           GPIO_PIN_B4,
33                           PIN_CONFIG_MUX_FUNC3);
34 }
35
36 // uart7_m0
37 RT_WEAK void uart7_m0_iomux_config(void)
38 {
39     HAL_PINCTRL_SetIOMUX(GPIO_BANK3,
40                           GPIO_PIN_C4,
41                           PIN_CONFIG_MUX_FUNC3);
42     HAL_PINCTRL_SetIOMUX(GPIO_BANK3,
43                           GPIO_PIN_C7,
44                           PIN_CONFIG_MUX_FUNC3);
45 }
46 #endif
```

图 2.5-5 修改配置函数

- 并在 `board/common/iomux_base.h` 中添加对应声明。
- 增加 `UART7_M0` 串口的配置后，还需要根据使用的具体情况修改串口波特率，根据一般使用情况，这里将调试串口的波特率修改为 115200。

```
56 #if defined(RT_USING_UART0)
57 RT_WEAK const struct uart_board g_uart0_board =
58 {
59     .baud_rate = UART_BR_1500000,
60     .dev_flag = ROCKCHIP_UART_SUPPORT_FLAG_DEFAULT,
61     .bufer_size = RT_SERIAL_RB_BUFSZ,
62     .name = "uart0",
63 };
64 #endif /* RT_USING_UART0 */
65
66 #if defined(RT_USING_UART7)
67 RT_WEAK const struct uart_board g_uart7_board =
68 {
69     .baud_rate = UART_BR_115200,
70     //.baud_rate = UART_BR_1500000,
71     .dev_flag = ROCKCHIP_UART_SUPPORT_FLAG_DEFAULT,
72     .bufer_size = RT_SERIAL_RB_BUFSZ,
73     .name = "uart7",
74 };
75 #endif /* RT_USING_UART7 */
```

图 2.5-6 修改配置函数

修改完上述配置后，我们成功的将 RT-Thread 的调试串口修改为底板适配的 `UART7_M0` 配置。

2.6 系统编译与部署流程

在 SDK 根目录中执行以下命令，即可自动编译 AMP 固件：

```
./build.sh amp
```

编译完成后，生成的固件文件为 `amp.img`，位于 `output/firmware/` 目录下。

```
Running 25-amp.sh - build_rththread RK AMP RTT_TARGET 3 rtt3 board/rk3562_evb1_lp4x/amp_defconfig succeeded.
FIT description: FIT source file for rockchip AMP
Created: Mon Apr 14 07:27:34 2025
Image 0 (amp3)
Description: bare-mental-core3
Created: Mon Apr 14 07:27:34 2025
Type: Firmware
Compression: uncompressed
Data Size: 159424 Bytes = 155.69 KiB = 0.15 MiB
Architecture: ARM
Load Address: 0x01800000
Hash algo: sha256
Hash value: 5e876941e2b25bdc83d926ca5d8a276f45618442588595338c12d8c733fcc8e6
Default Configuration: 'conf'
Configuration 0 (conf)
Description: Rockchip AMP images
Kernel: unavailable
Loadables: amp3
Running 25-amp.sh - amp amp succeeded.
```

图 2.6-1 编译界面

查看制作好的镜像。

```
hzh@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/firmware$ ls -l
total 192
-rw-rw-r-- 1 hzh hzh 197632 May 12 09:42 amp.img
lrwxrwxrwx 1 hzh hzh 26 May 12 09:41 boot.img -> ../../kernel-5.10/boot.img
lrwxrwxrwx 1 hzh hzh 44 May 12 09:38 MiniLoaderAll.bin -> ../../u-boot/rk3562_spl_loader_v1.07.106.bin
lrwxrwxrwx 1 hzh hzh 11 May 12 09:38 misc.img -> ../misc.img
lrwxrwxrwx 1 hzh hzh 22 May 12 09:38 uboot.img -> ../../u-boot/uboot.img
hzh@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/firmware$
```

图 2.6-2 查看镜像

将已添加 AMP 分区的分区表文件 `device/rockchip/rk3562/parameter-buildroot-fit.txt` 从 Ubuntu 系统拷贝到 PC 主机，并导入至烧录软件中，此时可以在界面中看到新增的 AMP 分区。

选择上一步编译好的固件。这里还需要选择新编译的 uboot 和 boot 固件以及 parameter 分区表文件，因为修改了 uboot 和 kernel 的配置，和设备树中调试串口的管脚，所以需要重新编译后下载最新的固件。

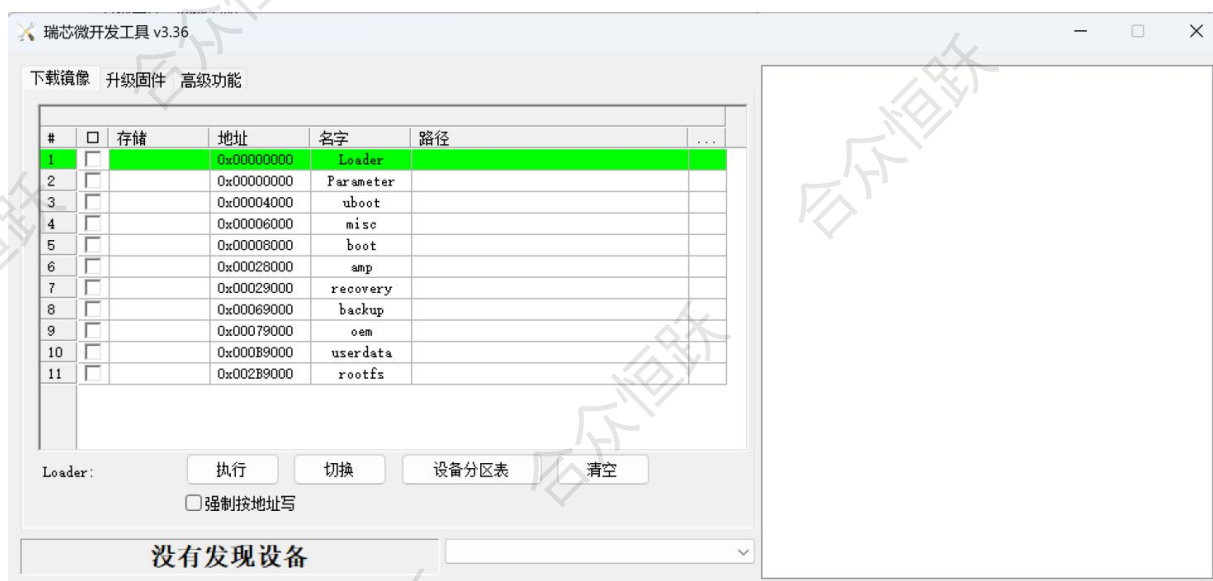


图 2.6-3 烧录软件界面

更新分区后，选择 uboot、boot、amp、parameter 进行烧录。



图 2.6-4 分区烧录

2.7 异构系统启动与测试

在上述过程中，已将 RT-Thread 的调试串口修改为 UART7_M0。因此，在实际接线时 also 需根据原理图连接 UART7_M0 对应的引脚。

对于 RK3562J-MINI 开发板，UART7_M0 的引脚位于 J30 排针座上，具体对应关系如下：

- TX: 第 35 引脚
- RX: 第 37 引脚
- GND: 第 39 引脚

请将开发板的 UART7 接口与 PC 的串口工具进行交叉连接（TX-RX，RX-TX，GND-GND）。连接完成后，在上位机打开串口调试工具，串口参数设置如下：

- 波特率：115200
- 数据位：8 位
- 停止位：1 位
- 校验位：无
- 流控：无

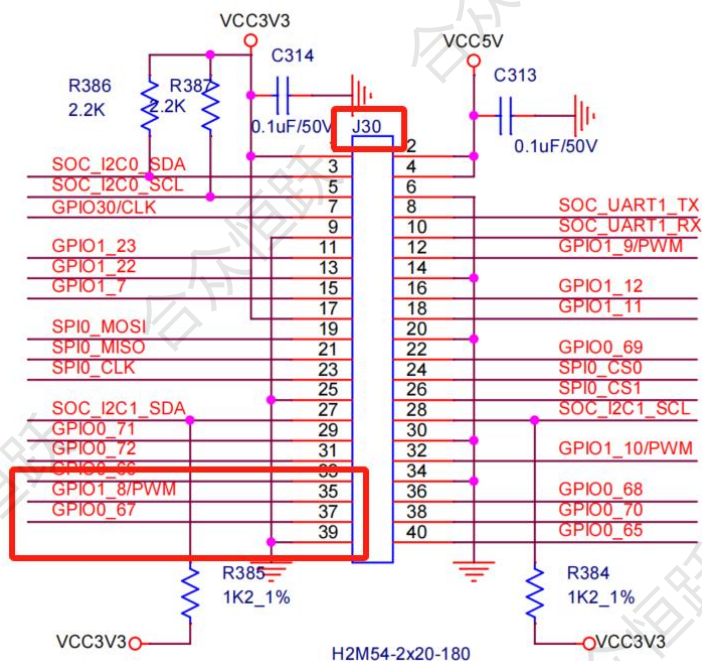


图 2.7-1 原理图

连接好 UART7 调试串口后，在烧录并启动板卡后，可以在串口调试助手中看到系统的默认启动打印信息。

```
msh />base_mem: BASE = 0x01800000, SIZE = 0x00800000
share_mem: BASE = 0x07800000, SIZE = 0x00400000
[HAL WARNING] HAL_GIC_Enable: invalid irq-32
[HAL WARNING] HAL_GIC_Enable: invalid irq-34
[HAL WARNING] HAL_GIC_Enable: invalid irq-36
[HAL WARNING] HAL_GIC_Enable: invalid irq-38
[HAL WARNING] HAL_GIC_Enable: invalid irq-40

\ | /
- RT -   Thread Operating System
/ | \   4.1.1 build Apr 14 2025 07:00:21
2006 - 2022 Copyright by RT-Thread team
Hi, this is RT-Thread!!
msh />
msh />
msh />
msh />
```

图 2.7-2 启动打印

输入内置命令可以查看部分系统信息，比如 `free` 查看内存使用，`list_device` 查看外设使用，其他命令可以自行尝试学习。


```
msh />
msh />free
total    : 8180352
used     : 7548
maximum  : 7548
available: 8172804
msh />
msh />
msh />list_device
device      type      ref count
-----
pin         Pin Device  0
uart7       Character Device  2
msh />
msh />
```

图 2.7-3 查看部分系统信息

2.8 核间通信配置

瑞芯微为多核异构系统提供了 RPMsg 协议标准框架方案，Linux Kernel 适配 RPMsg，RTOS 和 Baremetal 适配 RPMsg-Lite。它定义了 AMP 系统中核与核之间进行通信时所使用的标准二进制接口。在 RPMsg 中，主-从核心通过中断和共享内存的方式进行通信，内存的管理由主核负责，在每个通信方向上都有 USED 和 AVAIL 两个缓冲区，这两个缓冲区可以按照 RPMsg 的消息格式分成块一块，由这些内存块可以链接成一个环。

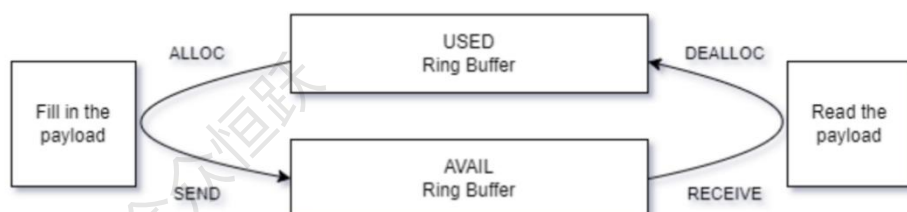


图 2.8-1 系统架构

因此当主核（Master Core）和从核（Remote Core）进行通信时：

- 1) Master Core 发送时，从 vring0(USED)中取得一块 buffer，再将消息按照 RPMsg 协议填充。
- 2) 将处理好的内存 buffer 链接到 vring1(AVAIL)。
- 3) 触发中断通知 Remote Core 有数据处理待处理。

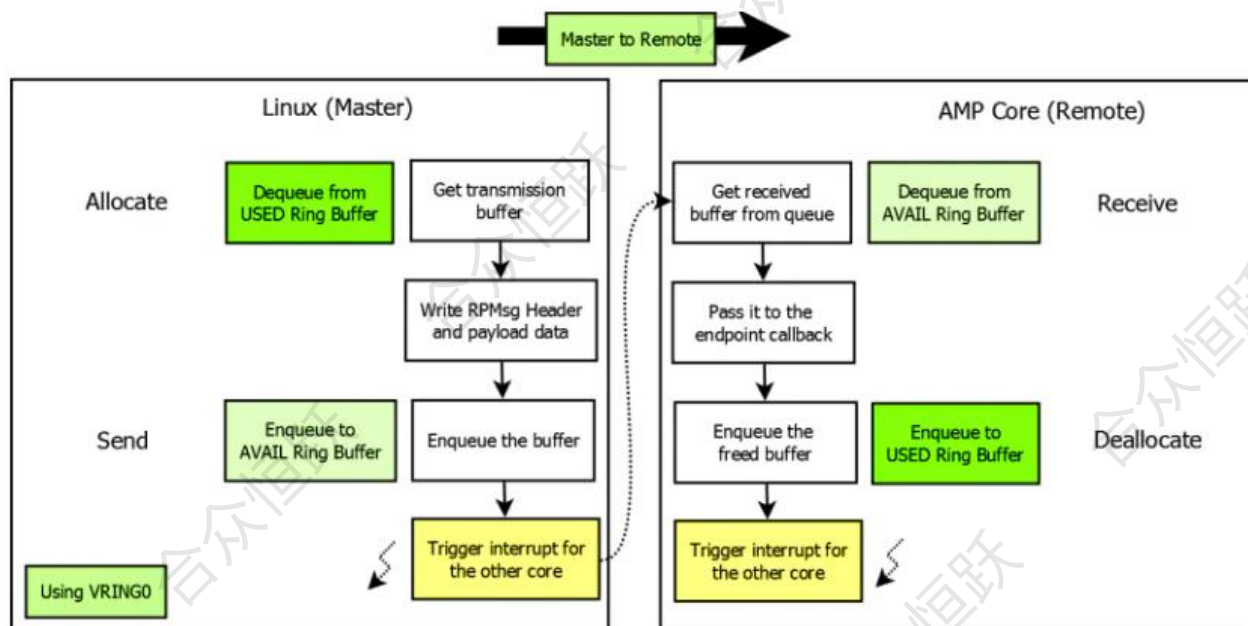


图 2.8-2 通信过程

当从核需要和主核进行通信时：

- 1) 从核根据队列从 vring1(AVAIL)中取得一块 buffer，再将消息按照 RPMsg 协议填充。
- 2) 将处理好的内存 buffer 链接到 vring0(USED)。
- 3) 触发中断通知 Master Core 有数据处理待处理。

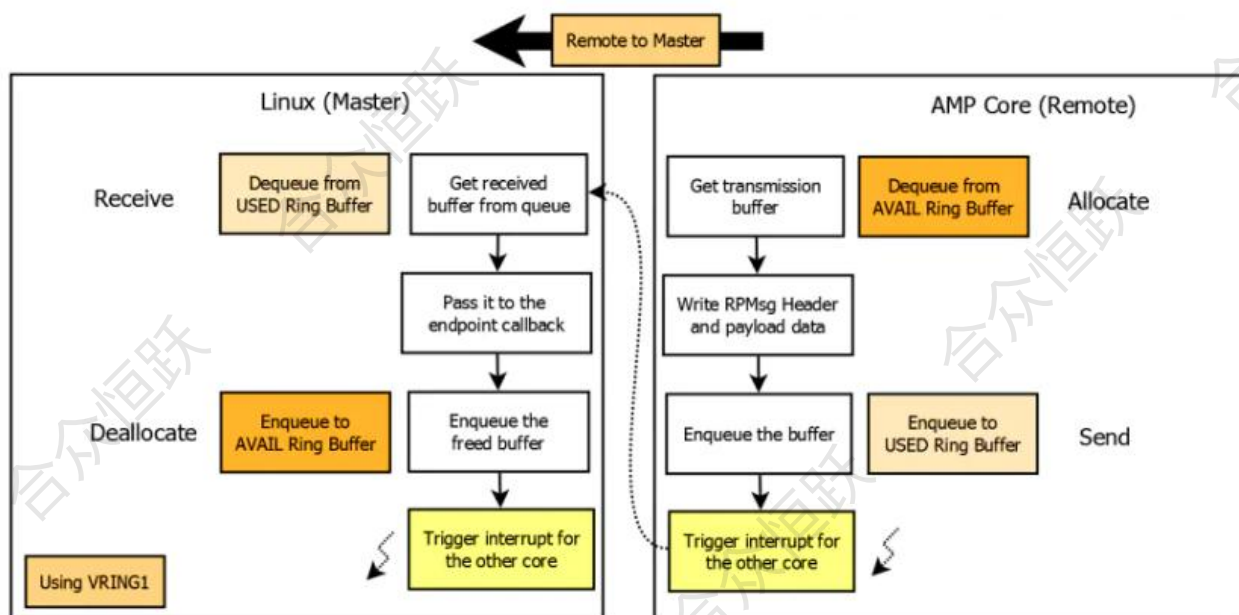


图 2.8-3 通信过程

完成消息传递后，释放使用的 buffer，并等待下一笔数据发送。从核发送时，与主核发送流程相反。通

信过程中的共享数据放在 vdev buffer 中。

RPMsg 每次发送的最大数据长度取决于 payload 长度，这个长度在 SDK 中默认为 512 Bytes，由于 RPMsg 还带有 16 Bytes 的数据头，因此一次性传输的最大数据量为 496 Bytes，详细可以参考 drivers/rpmsg/virtio_rpmsg_bus.c。

2.8.1 增加 Kernel 的 rpmsg 支持

在 kernel 配置中增加 RPMSG_ROCKCHIP_CDEV 配置，开启配置界面和保存流程参考上述流程，这里不再赘述。

```
Symbol: RPMSG_ROCKCHIP_CDEV [=y]
Type : tristate
Defined at drivers/rpmsg/Kconfig:87
Prompt: Rockchip RPMsg Cdev Test
Depends on: RPMSG_ROCKCHIP_MBOX [=y] || RPMSG_ROCKCHIP_SOFTIRQ [=n]
Location:
-> Device Drivers
(1) -> Rpmsg drivers
```

图 2.8-4 修改配置

增加配置后保存配置，并重新编译烧录内核。

2.8.2 增加 RT-Thread 的 rpmsg 支持

在 RT-Thread 配置中增加 RT_USING_RPMSG_CDEV_THREAD_TEST 配置，开启配置界面和保存流程参考上述流程，这里不再赘述。

```
There is no help available for this option.
Symbol: RT_USING_RPMSG_CDEV_THREAD_TEST [=y]
Type : boolean
Prompt: Enable cdev rpmsg tty test
Location:
-> RT-Thread bsp test case
-> RT-Thread Common Test case
-> Enable BSP Common TEST (RT_USING_COMMON_TEST [=y])
Defined at ../../common/tests/Kconfig:261
Depends on: RT_USING_COMMON_TEST [=y] && RT_USING_RPMSG_LITE [=y]
```

图 2.8-5 修改配置

增加 kernel 和 RT-Thread 的 rpmsg 的支持后，重新编译烧录并等待板卡启动。

观察 RT-Thread 串口打印，相比之前增加 rpmsg 相关的打印。

```
\ | /  
- RT -   Thread Operating System  
/ | \  
4.1.1 build Apr 17 2025 11:51:56  
2006 - 2022 Copyright by RT-Thread team  
rpsmsg cdev init  
rpsmsg remote: remote core cpu_id-3  
rpsmsg remote: shmem_base-0x7c000000 shmem_end-8100000  
rpsmsg remote: link up! link_id-0x3  
rpsmsg initialization completed  
Hi, this is RT-Thread!!  
msh />  
msh />  
msh />
```

图 2.8-6 增加 rpsmsg 相关的打印

登录 Linux 系统后，查询 rpsmsg 节点是否存在，执行 `ls /dev/rpsmsg_tty_3003`，可以看到新增的 rpsmsg 字符设备。

```
root@rk3562-buildroot:~#  
root@rk3562-buildroot:~# ls /dev/r  
ram0          rfskill        rpsmsg_tty_3003  
random        rga  
root@rk3562-buildroot:~# ls /dev/rpsmsg_tty_3003  
/dev/rpsmsg_tty_3003  
root@rk3562-buildroot:~#  
root@rk3562-buildroot:~#  
root@rk3562-buildroot:~#
```

图 2.8-7 查看新增字符设备

在 RT-Thread 和 Linux 端都可以看到 rpsmsg 的支持后，使用命令进行测试。在 RT-Thread 端执行 `rpsmsg_cdev_start`，然后切换到 Linux 端执行 `rpsmsg_demo/dev/rpsmsg_tty_3003`，可以看到测试命令发送的 hello 消息，并收到 RT-Thread 回应的消息 Rockchip rpsmsg linux test。

相关代码参考 `rtos/bsp/rockchip/rk3562-32/applications/rpsmsg_demo.c`。

```
root@rk3562-buildroot:~# rpsmsg_demo /dev/rpsmsg_tty_3003  
成功打开设备: /dev/rpsmsg_tty_3003  
开始自动发送消息...  
已发送 7 字节: hello 0  
收到: Rockchip rpsmsg linux test!  
已发送 7 字节: hello 1  
收到: Rockchip rpsmsg linux test!  
已发送 7 字节: hello 2  
收到: Rockchip rpsmsg linux test!  
已发送 7 字节: hello 3  
收到: Rockchip rpsmsg linux test!  
已发送 7 字节: hello 4  
收到: Rockchip rpsmsg linux test!  
已发送 7 字节: hello 5  
收到: Rockchip rpsmsg linux test!
```

图 2.8-8 Linux 端打印界面

在 RT-Thread 端可以看到 Linux 端发送来的消息。

```
msh />rpmsg_cdev_start
rpmsg test thread started
msh />
msh />
msh />rpmsg remote: master_ept_id-0x400 rx_msg: hello 0
rpmsg remote: master_ept_id-0x400 rx_msg: hello 1
rpmsg remote: master_ept_id-0x400 rx_msg: hello 2
rpmsg remote: master_ept_id-0x400 rx_msg: hello 3
rpmsg remote: master_ept_id-0x400 rx_msg: hello 4
rpmsg remote: master_ept_id-0x400 rx_msg: hello 5
```

图 2.8-9 RT-Thread 端打印界面